UNIVERSIDAD ADVENTISTA DEL PLATA
FACULTAD DE CIENCIAS ECONÓMICAS Y DE LA ADMINISTRACIÓN
ALGORITMOS Y ESTRUCTURA DE DATOS

LISTA DE EJERCICIOS Nº 6 — ESTRUCTURAS DINÁMICAS Y TADs (U6)

Sección 1: Listas enlazadas

1. Crear y recorrer una lista enlazada simple*.
    a. Crear una clase Nodo que contenga:
        i. Un valor de tipo String.
        ii. Una referencia al siguiente nodo.
    b. Crear tres nodos y enlazarlos entre sí.
    c. Recorrer la lista desde el primer nodo y mostrar todos sus valores.
    d. Modificar el valor de uno de los nodos y volver a mostrar la lista.
2. Crear una clase Lista para administrar una lista enlazada*.
    a. Crear una clase Lista que tenga una referencia al primer nodo.
    b. Crear un método agregar(String valor) que permita agregar un nuevo nodo al final de la lista.
    c. Crear un método mostrar() que recorra la lista y muestre todos sus elementos.
    d. Crear una lista de nombres de jugadores y agregar al menos 5 elementos.
    e. Mostrar todos los elementos de la lista.
3. Crear operaciones sobre una lista enlazada.
    a. A partir de la clase Lista del ejercicio anterior:
        i. Crear un método para determinar si un elemento se encuentra en la lista.
        ii. Crear un método que indique cuántos elementos contiene la lista.
        iii. Crear un método que permita eliminar un elemento de la lista.
        iv. Probar las operaciones agregando, buscando y eliminando diferentes elementos.
4. Crear una lista enlazada doble.
    a. Modificar la clase Nodo para que cada nodo tenga referencias al nodo siguiente y al nodo anterior.
    b. Crear una lista con al menos 5 elementos.
    c. Recorrer la lista desde el primer elemento hasta el último.
    d. Recorrer la lista desde el último elemento hasta el primero.
    e. Mostrar los elementos en ambos sentidos.

Sección 2: Referencias y memoria

5. Analizar el comportamiento de referencias entre objetos*.
    a. Considerar la siguiente clase y el código que sigue:

```
class Personaje {
    String nombre;
}
```

```
Personaje a = new Personaje();
a.nombre = "Link";
Personaje b = a;
b.nombre = "Mario";
```

    b. ¿Qué valor tendrá a.nombre al finalizar el programa?
    c. ¿Qué valor tendrá b.nombre?
    d. Explicar mediante un esquema qué representan a y b y a qué objeto hacen referencia.
    e. Modificar el ejemplo para crear dos objetos independientes y observar la diferencia.

Sección 3: Tipos Abstractos de Datos

6. Crear un TAD para representar una colección de elementos.
    a. Definir mediante una interfaz las operaciones que debería ofrecer una estructura que permita:
        i. Agregar un elemento.
        ii. Eliminar un elemento.
        iii. Consultar si un elemento existe.
        iv. Obtener la cantidad de elementos.
    b. Crear una clase que implemente dicha interfaz utilizando una lista enlazada.
    c. Implementar las operaciones definidas.
    d. Crear un programa que utilice la estructura a través de las operaciones definidas por el TAD.

Sección 4: Pilas

7. Implementar una pila utilizando una lista enlazada*.
    a. Crear una estructura que permita administrar elementos siguiendo el comportamiento LIFO (Last In, First Out).
        i. Crear las operaciones push y pop.
        ii. Agregar varios elementos a la pila.
        iii. Mostrar el elemento que se encuentra en la cima.
        iv. Extraer elementos de la pila y mostrar el orden en que son extraídos.
        v. Probar qué ocurre cuando se intenta extraer un elemento de una pila vacía.
8. Utilizar una pila para invertir una secuencia.

a. Solicitar al usuario una palabra.
b. Agregar cada uno de sus caracteres a una pila.
c. Extraer los caracteres de la pila para construir una nueva cadena.
d. Mostrar la cadena original y la cadena resultante.
e. Ejemplo:

```
Ingrese una palabra: programar

Original: programar
Resultado: ramargorp
```

Sección 5: Colas

9. Implementar una cola utilizando una lista enlazada*.
    a. Crear una estructura que permita administrar elementos siguiendo el comportamiento FIFO (First In, First Out).
        i. Crear las operaciones necesarias para agregar y retirar elementos.
        ii. Agregar varios jugadores a una cola de espera.
        iii. Retirar jugadores de la cola y mostrar el orden en que son atendidos.
        iv. Mostrar quién es el próximo jugador que será atendido sin retirarlo de la cola.
        v. Probar qué ocurre cuando se intenta retirar un elemento de una cola vacía.
10. Simular un sistema de atención utilizando una cola.
    a. Una tienda de videojuegos tiene una única fila de atención.
        i. Crear una cola de clientes.
        ii. Permitir agregar clientes a la fila.
        iii. Atender clientes en el mismo orden en que llegaron.
        iv. Mostrar en todo momento quién es el próximo cliente a atender.
        v. Mostrar los clientes que todavía permanecen en la fila.
        vi. Permitir que el usuario continúe agregando y atendiendo clientes hasta seleccionar la opción de finalizar.

Sección 6: Problemas integradores

11. Administrar una partida mediante estructuras dinámicas.
    a. Crear un programa para administrar los jugadores de una partida de videojuegos.
        i. Crear una clase Jugador con nombre y nivel.
        ii. Crear una lista enlazada para almacenar los jugadores.
        iii. Permitir agregar y eliminar jugadores.
        iv. Permitir buscar un jugador por su nombre.
        v. Crear una cola para representar el orden en que los jugadores esperan para ingresar a una partida.
        vi. Crear una pila para almacenar las últimas acciones realizadas durante la partida.

vii. Mostrar ejemplos de uso de cada estructura.

12. Analizar y comparar las estructuras utilizadas.
    a. Para cada una de las siguientes estructuras:
        i. Array
        ii. ArrayList
        iii. Lista enlazada
        iv. Pila
        v. Cola
    b. Indicar cómo se almacenan y organizan los elementos.
    c. Indicar cómo se agrega un elemento.
    d. Indicar cómo se elimina un elemento.
    e. Indicar qué elemento se obtiene primero al retirar elementos.